Freestanding constexpr containers and constexpr exception types

Document number: P3295R0
Date: 2024-05-18
Reply-to: Ben Craig <ben dot craig at gmail dot com>
Audience: SG7 Compile Time Programming, Library Evolution Working Group

Changes from previous revisions

R0

First revision!

Introduction

This proposal helps enable P2996 reflection and more advanced metaprogramming for freestanding environments. Specifically, this allows `std::vector`, `std::string`, and `std::allocator` to be used in constant evaluated contexts in freestanding.

As a first approximation, this makes the `vector` and `string` facilities `consteval` in freestanding. Freestanding implementations that know more about their targets are permitted to provide the full `constexpr` facilities if they wish.

This paper is attempting to modify the minimum set of facilities possible to freestanding in order to enable reflection.

Motivation and Scope

Freestanding applications are often unable to allocate memory or throw exceptions at runtime. No such restriction exists while building the application though.

Reflection in P2996R2 contains many `consteval` functions that return `vector<meta::info>`, such as `members_of`. Those facilities are valuable in freestanding and hosted alike.

This paper would "only" make `vector` and `allocator` consteval on freestanding if it reasonably could as a means to limit scope, but `vector` has functions that throw `length_error` and `out_of_range`. `length_error` and `out_of_range` are not in freestanding prior to this paper, so mentioning them renders the program ill-formed, even in a constant evaluation context. This paper therefore makes those exception types `constexpr` on hosted, and `consteval` on freestanding. Making these exception types `constexpr` is likely to be a benefit for long-term reflection error handling plans (e.g. P3068 Allowing exception throwing in constant-evaluation).

Similarly, `length_error` and `out_of_range` mention `string`, which is also not yet in freestanding. Therefore the scope expands to `string`. Fortunately, `char_traits` and most of `string_view` are already freestanding.

The *freestanding-deleted* methods of `string_view` will need to be made `consteval` on freestanding in order to support `string`s use cases.

Hosted `constexpr` additions:

- `exception`
- `logic_error`
- `length_error`
- `out_of_range`

freestanding-consteval additions:

- `logic_error`
- `length_error`
- `out_of_range`
- `allocator`
- `string_view` freestanding-deleted members
- `string`
- `vector`

Design Considerations

Freestanding all the `constexpr` things?

In P2268 Freestanding Roadmap, a strategy is discussed of making everything that is `constexpr` in the standard library available at compile time via `consteval`, and those facilities that are also suitable at runtime to be marked `constexpr` and `// freestanding`. That is still the desired goal. Reaching

that goal requires a lot of research and testing. There are likely other, non-`constexpr` facilities that the `constexpr` facilities rely on that would need to be audited as well.

This paper has chosen to prioritize the reflection subset of that work, rather than block progress on freestanding reflection.

Performance regression when porting from freestanding to hosted

`consteval` functions are never evaluated during runtime, but `constexpr` functions are sometime evaluated during runtime. A freestanding library using a `consteval` facility may end up "downgrading" some code from compile time to runtime during a port to a hosted environment due to the change to `constexpr`. This can result in worse performance due to the runtime function calls and allocations that were previously compile time function calls and allocations.

In this unoptimized example, we get runtime calls to an allocating constructor and deallocating destructor in the `constexpr` version compared to the `consteval` version. For this example, the difference disappears when optimizations are enabled, but that won't always be the case.

Non-transient `constexpr` allocations

There is a desire from some in the committee to allow constant evaluated `vectors` and `strings` persist into runtime. This is referred to as non-transient `constexpr` allocations. This paper doesn't propose such facilities, but the paper is trying to avoid causing issues for those future papers.

If we get non-transient `constexpr` allocations in the future, then we can still make that work for freestanding. Constructors and mutating methods could remain `consteval`, with non-mutating methods made `constexpr`. This would avoid allocations and deallocations at runtime, while allowing the constant compile time object to be observed.

`constexpr logic_errors`

Philosophy

Various methods (`vector::at`, `vector::reserve`, `string::compare`, etc) throw exceptions in the `logic_error` hierarchy. For short term freestanding purposes, we could choose not to add the `logic_error` hierarchy, and instead have implementations `#if` that code out, and put something else in that spot that would cause the code to fail constant evaluation. However, there are proposals to permit throws during constant evaluation (P3068 Allowing exception throwing in constant-evaluation), and there is a desire to use this facility as part of post-C++26 P2996 reflection. So rather than have implementers add a hack only to remove it in the near future, we'll add the `logic_errors` we need to `consteval` freestanding, and mark them as `constexpr` in hosted.

There are some other reasons to make the `logic_error` hierarchy `constexpr`. There's the general "`constexpr` all the things" motivation. Some developers are also interested in having `variant` and `expected` objects with `logic_error` alternatives as a way to manage errors in `constexpr` contexts.

We could also choose to make the entirety of `<stdexcept>` `consteval` in freestanding. The different classes are mostly identical in terms of functionality.

Technical challenges

There are some reasons why making `logic_error` `constexpr` could be challenging.

The libc++ implementation of `logic_error` includes a reference counted string. The reference counting currently uses atomic operations, though it doesn't use `std::atomic` specifically. That implementation would need to conditionally use non-atomic operations during constant evaluation.

The libstdc++ implementation uses a copy-on-write (COW) string. Portions of the implementation of the `logic_error` hierarchy are in the library rather than headers. One of the COW string implementations use atomic operations for the reference counting implementation, but not specifically `std::atomic`

MSSTL appears to perform a deep copy of the strings in `logic_error`. It does not appear to involve atomics. Some of its implementation appears to be in the library, and not in headers.

P3037R1 `constexpr std::shared_ptr` also discusses reference counting in `constexpr`. SG7 Compile-time programming was fine with reference counting at compile time in the Tokyo 2024 meeting.

Experience

No implementation experience yet.

Wording

This paper's wording is based on the current working draft N4981.

Changes in [freestanding.item]

TODO: define what "freestanding-consteval" means. It is implementation defined whether "freestanding-consteval" functions are `constexpr` or `consteval` on freestanding implementations. Destructors, and maybe all virtual functions will need to remain `constexpr`, even on a freestanding-consteval class.

Changes in [compliance]

Drafting note: `<string>` and `<memory>` are already freestanding headers.
Add new rows to the "C++ headers for freestanding implementations" table:

Subclause		Header(s)
[…]	[…]	[…]
?.? [std.exceptions]	**Exception classes**	**<stdexcept>**
[…]	[…]	[…]
?.? [vector.syn]	**Header <vector> synopsis**	**<vector>**
[…]	[…]	[…]

Changes in [version.syn]

Instructions to the editor:
Add the following macros to [version.syn]:

```
#define __cpp_lib_constexpr_exception 20XXXXL // freestanding, also in <exception>, <stdexcept>
#define __cpp_lib_freestanding_consteval_allocator 20XXXXL // freestanding, also in <memory>
#define __cpp_lib_freestanding_consteval_string 20XXXXL // freestanding, also in <string>
#define __cpp_lib_freestanding_stdexcept 20XXXXL // freestanding, also in <stdexcept>
#define __cpp_lib_freestanding_vector 20XXXXL // freestanding, also in <vector>
```

Changes in [exception]

Instructions to the editor:
Please make the following edits to [exception]:

```
namespace std {
  class exception {
  public:
    constexpr exception() noexcept;
    constexpr exception(const exception&) noexcept;
    constexpr exception& operator=(const exception&) noexcept;
    constexpr virtual ~exception();
    constexpr virtual const char* what() const noexcept;
  };
}
/* ... */
constexpr exception(const exception& rhs) noexcept;
constexpr exception& operator=(const exception& rhs) noexcept;

/* ... */
constexpr virtual ~exception();

/* ... */
constexpr virtual const char* what() const noexcept;
```

Changes in [stdexcept.syn]

Instructions to the editor:
Please make the following edits to [stdexcept.syn]:

```
namespace std {
  class logic_error; // freestanding-consteval
    class domain_error;
    class invalid_argument;
    class length_error; // freestanding-consteval
    class out_of_range; // freestanding-consteval
  class runtime_error;
    class range_error;
    class overflow_error;
```

```
    class underflow_error;
}
```

Changes in [logic.error]

Instructions to the editor:
Please make the following edits to [logic.error]:

```
namespace std {
  class logic_error : public exception {
  public:
    constexpr explicit logic_error(const string& what_arg);
    constexpr explicit logic_error(const char* what_arg);
    constexpr logic_error& operator=(const logic_error&) noexcept;
    constexpr virtual ~logic_error();
    constexpr virtual const char* what() const noexcept;
  };
}
/*...*/
constexpr logic_error(const string& what_arg);
/*...*/
constexpr logic_error(const char* what_arg);
```

Changes in [length.error]

Instructions to the editor:
Please make the following edits to [length.error]:

```
namespace std {
  class length_error : public logic_error {
  public:
    constexpr explicit length_error(const string& what_arg);
    constexpr explicit length_error(const char* what_arg);
    constexpr length_error& operator=(const length_error&) noexcept;
    constexpr virtual ~length_error();
    constexpr virtual const char* what() const noexcept;
  };
}
/*...*/
constexpr length_error(const string& what_arg);
/*...*/
constexpr length_error(const char* what_arg);
```

Changes in [out.of.range]

Instructions to the editor:
Please make the following edits to [out.of.range]:

```
namespace std {
  class out_of_range : public logic_error {
  public:
    constexpr explicit out_of_range(const string& what_arg);
    constexpr explicit out_of_range(const char* what_arg);
    constexpr out_of_range& operator=(const out_of_range&) noexcept;
    constexpr virtual ~out_of_range();
    constexpr virtual const char* what() const noexcept;
  };
}
/*...*/
constexpr out_of_range(const string& what_arg);
/*...*/
constexpr out_of_range(const char* what_arg);
```

Changes in [memory.syn]

Instructions to the editor:
Please make the following edit to [memory.syn]:

```
// [default.allocator], the default allocator
template<class T> class allocator; // freestanding-consteval
template<class T, class U>
  constexpr bool operator==(const allocator<T>&, const allocator<U>&) noexcept; // freestanding-consteval
```

Changes in [string.view.template.general]

Instructions to the editor:
Please make the following edit to [string.view.template.general]:

```
  // [string.view.access], element access
  constexpr const_reference operator[](size_type pos) const;
  constexpr const_reference at(size_type pos) const;                  // freestanding-deleted consteval
  constexpr const_reference front() const;
  constexpr const_reference back() const;
  constexpr const_pointer data() const noexcept;

  // [string.view.modifiers], modifiers
  constexpr void remove_prefix(size_type n);
  constexpr void remove_suffix(size_type n);
  constexpr void swap(basic_string_view& s) noexcept;

  // [string.view.ops], string operations
  constexpr size_type copy(charT* s, size_type n,
                           size_type pos = 0) const;                  // freestanding-deleted consteval

  constexpr basic_string_view substr(size_type pos = 0,
                                     size_type n = npos) const;       // freestanding-deleted consteval

  constexpr int compare(basic_string_view s) const noexcept;
  constexpr int compare(size_type pos1, size_type n1,
                        basic_string_view s) const;                   // freestanding-deleted consteval
  constexpr int compare(size_type pos1, size_type n1, basic_string_view s,
                        size_type pos2, size_type n2) const;          // freestanding-deleted consteval
  constexpr int compare(const charT* s) const;
  constexpr int compare(size_type pos1, size_type n1,
                        const charT* s) const;                        // freestanding-deleted consteval
  constexpr int compare(size_type pos1, size_type n1, const charT* s,
                        size_type n2) const;                          // freestanding-deleted consteval
```

Changes in [string.syn]

Instructions to the editor:
Please make the following edits to [string.syn]:

```
// [basic.string], basic_string
template<class charT, class traits = char_traits<charT>, class Allocator = allocator<charT>>
  class basic_string; // freestanding-consteval

template<class charT, class traits, class Allocator>
  constexpr basic_string<charT, traits, Allocator>
    operator+(const basic_string<charT, traits, Allocator>& lhs,
              const basic_string<charT, traits, Allocator>& rhs); // freestanding-consteval
template<class charT, class traits, class Allocator>
  constexpr basic_string<charT, traits, Allocator>
    operator+(basic_string<charT, traits, Allocator>&& lhs,
              const basic_string<charT, traits, Allocator>& rhs); // freestanding-consteval
template<class charT, class traits, class Allocator>
  constexpr basic_string<charT, traits, Allocator>
    operator+(const basic_string<charT, traits, Allocator>& lhs,
              basic_string<charT, traits, Allocator>&& rhs); // freestanding-consteval
template<class charT, class traits, class Allocator>
  constexpr basic_string<charT, traits, Allocator>
    operator+(basic_string<charT, traits, Allocator>&& lhs,
              basic_string<charT, traits, Allocator>&& rhs); // freestanding-consteval
```

```
template<class charT, class traits, class Allocator>
  constexpr basic_string<charT, traits, Allocator>
    operator+(const charT* lhs,
              const basic_string<charT, traits, Allocator>& rhs); // freestanding-consteval
template<class charT, class traits, class Allocator>
  constexpr basic_string<charT, traits, Allocator>
    operator+(const charT* lhs,
              basic_string<charT, traits, Allocator>&& rhs); // freestanding-consteval
template<class charT, class traits, class Allocator>
  constexpr basic_string<charT, traits, Allocator>
    operator+(charT lhs,
              const basic_string<charT, traits, Allocator>& rhs); // freestanding-consteval
template<class charT, class traits, class Allocator>
  constexpr basic_string<charT, traits, Allocator>
    operator+(charT lhs,
              basic_string<charT, traits, Allocator>&& rhs); // freestanding-consteval
template<class charT, class traits, class Allocator>
  constexpr basic_string<charT, traits, Allocator>
    operator+(const basic_string<charT, traits, Allocator>& lhs,
              const charT* rhs); // freestanding-consteval
template<class charT, class traits, class Allocator>
  constexpr basic_string<charT, traits, Allocator>
    operator+(basic_string<charT, traits, Allocator>&& lhs,
              const charT* rhs); // freestanding-consteval
template<class charT, class traits, class Allocator>
  constexpr basic_string<charT, traits, Allocator>
    operator+(const basic_string<charT, traits, Allocator>& lhs,
              charT rhs); // freestanding-consteval
template<class charT, class traits, class Allocator>
  constexpr basic_string<charT, traits, Allocator>
    operator+(basic_string<charT, traits, Allocator>&& lhs,
              charT rhs); // freestanding-consteval
template<class charT, class traits, class Allocator>
  constexpr basic_string<charT, traits, Allocator>
    operator+(const basic_string<charT, traits, Allocator>& lhs,
              type_identity_t<basic_string_view<charT, traits>> rhs); // freestanding-consteval
template<class charT, class traits, class Allocator>
  constexpr basic_string<charT, traits, Allocator>
    operator+(basic_string<charT, traits, Allocator>&& lhs,
              type_identity_t<basic_string_view<charT, traits>> rhs); // freestanding-consteval
template<class charT, class traits, class Allocator>
  constexpr basic_string<charT, traits, Allocator>
    operator+(type_identity_t<basic_string_view<charT, traits>> lhs,
              const basic_string<charT, traits, Allocator>& rhs); // freestanding-consteval
template<class charT, class traits, class Allocator>
  constexpr basic_string<charT, traits, Allocator>
    operator+(type_identity_t<basic_string_view<charT, traits>> lhs,
              basic_string<charT, traits, Allocator>&& rhs); // freestanding-consteval

template<class charT, class traits, class Allocator>
  constexpr bool
    operator==(const basic_string<charT, traits, Allocator>& lhs,
               const basic_string<charT, traits, Allocator>& rhs) noexcept; // freestanding-consteval
template<class charT, class traits, class Allocator>
  constexpr bool operator==(const basic_string<charT, traits, Allocator>& lhs,
                            const charT* rhs); // freestanding-consteval

template<class charT, class traits, class Allocator>
  constexpr see below operator<=>(const basic_string<charT, traits, Allocator>& lhs,
                                  const basic_string<charT, traits, Allocator>& rhs) noexcept; // freestanding-consteval
template<class charT, class traits, class Allocator>
  constexpr see below operator<=>(const basic_string<charT, traits, Allocator>& lhs,
                                  const charT* rhs); // freestanding-consteval

// [string.special], swap
template<class charT, class traits, class Allocator>
  constexpr void
    swap(basic_string<charT, traits, Allocator>& lhs,
         basic_string<charT, traits, Allocator>& rhs)
      noexcept(noexcept(lhs.swap(rhs))); // freestanding-consteval

// [string.io], inserters and extractors
template<class charT, class traits, class Allocator>
  basic_istream<charT, traits>&
    operator>>(basic_istream<charT, traits>& is,
               basic_string<charT, traits, Allocator>& str);
template<class charT, class traits, class Allocator>
  basic_ostream<charT, traits>&
```

```
    operator<<(basic_ostream<charT, traits>& os,
               const basic_string<charT, traits, Allocator>& str);
  template<class charT, class traits, class Allocator>
    basic_istream<charT, traits>&
      getline(basic_istream<charT, traits>& is,
              basic_string<charT, traits, Allocator>& str,
              charT delim);
  template<class charT, class traits, class Allocator>
    basic_istream<charT, traits>&
      getline(basic_istream<charT, traits>&& is,
              basic_string<charT, traits, Allocator>& str,
              charT delim);
  template<class charT, class traits, class Allocator>
    basic_istream<charT, traits>&
      getline(basic_istream<charT, traits>& is,
              basic_string<charT, traits, Allocator>& str);
  template<class charT, class traits, class Allocator>
    basic_istream<charT, traits>&
      getline(basic_istream<charT, traits>&& is,
              basic_string<charT, traits, Allocator>& str);

  // [string.erasure], erasure
  template<class charT, class traits, class Allocator, class U = charT>
    constexpr typename basic_string<charT, traits, Allocator>::size_type
      erase(basic_string<charT, traits, Allocator>& c, const U& value); // freestanding-consteval
  template<class charT, class traits, class Allocator, class Predicate>
    constexpr typename basic_string<charT, traits, Allocator>::size_type
      erase_if(basic_string<charT, traits, Allocator>& c, Predicate pred); // freestanding-consteval

  // basic_string typedef-names
  using string    = basic_string<char>; // freestanding
  using u8string  = basic_string<char8_t>; // freestanding
  using u16string = basic_string<char16_t>; // freestanding
  using u32string = basic_string<char32_t>; // freestanding
  using wstring   = basic_string<wchar_t>; // freestanding
/* ... */
  inline namespace literals {
    inline namespace string_literals {
      // [basic.string.literals], suffix for basic_string literals
      constexpr string    operator""s(const char* str, size_t len); // freestanding-consteval
      constexpr u8string  operator""s(const char8_t* str, size_t len); // freestanding-consteval
      constexpr u16string operator""s(const char16_t* str, size_t len); // freestanding-consteval
      constexpr u32string operator""s(const char32_t* str, size_t len); // freestanding-consteval
      constexpr wstring   operator""s(const wchar_t* str, size_t len); // freestanding-consteval
    }
  }
```

Changes in [vector.syn]

Instructions to the editor:
Please make the following edits to [vector.syn]:

```
  // [vector], class template vector // freestanding-consteval
  template<class T, class Allocator = allocator<T>> class vector; // freestanding-consteval

  template<class T, class Allocator>
    constexpr bool operator==(const vector<T, Allocator>& x, const vector<T, Allocator>& y); // freestanding-consteval
  template<class T, class Allocator>
    constexpr synth-three-way-result<T> operator<=>(const vector<T, Allocator>& x,
                                                    const vector<T, Allocator>& y); // freestanding-consteval

  template<class T, class Allocator>
    constexpr void swap(vector<T, Allocator>& x, vector<T, Allocator>& y)
      noexcept(noexcept(x.swap(y))); // freestanding-consteval

  // [vector.erasure], erasure
  template<class T, class Allocator, class U = T>
    constexpr typename vector<T, Allocator>::size_type
      erase(vector<T, Allocator>& c, const U& value); // freestanding-consteval
  template<class T, class Allocator, class Predicate>
    constexpr typename vector<T, Allocator>::size_type
      erase_if(vector<T, Allocator>& c, Predicate pred); // freestanding-consteval
```

```
  namespace pmr {
    template<class T>
      using vector = std::vector<T, polymorphic_allocator<T>>;
  }

  // [vector.bool], specialization of vector for bool
  // [vector.bool.pspc], partial class template specialization vector<bool, Allocator>
  template<class Allocator>
    class vector<bool, Allocator>;  // freestanding-consteval
```